

Package ‘MultFourier’

September 24, 2026

Type Package

Title p-Values of Multinomial Goodness-of-Fit Tests by Fourier Series Inversion

Version 1.0

Author Charles Thraves, Pablo Rivas, Felipe Subiabre

Maintainer Pablo Rivas <pablo.rivas.g@ug.uchile.cl>

Description Computes p-values of multinomial goodness-of-fit tests based on an additive statistic: the log-likelihood ratio, Pearson's chi-squared, the Cressie-Read power divergence, and the exact multinomial test. The main function, `pval_fourier()`, writes the p-value as a Fourier series in the moment-generating function of the statistic and sums it until an adaptive stopping rule is met, which makes small p-values of large instances computable. The function `pval_exact()` computes the exact p-value by branch pruning with bounds from a dynamic program, and `pval_flexible()` chooses between the two according to the size of the instance.

License GPL (>= 2)

Encoding UTF-8

Suggests knitr,
rmarkdown,
ggplot2

VignetteBuilder knitr

Config/roxygen2/version 8.1.0

R topics documented:

MultFourier-package	2
print.multfourier	3
pval_exact	3
pval_flexible	5
pval_fourier	7

Index	11
--------------	-----------

Description

Computes p-values of multinomial goodness-of-fit tests based on an additive statistic: the log-likelihood ratio, Pearson's chi-squared, the Cressie-Read power divergence, and the exact multinomial test. The main function, `pval_fourier()`, writes the p-value as a Fourier series in the moment-generating function of the statistic and sums it until an adaptive stopping rule is met, which makes small p-values of large instances computable. The function `pval_exact()` computes the exact p-value by branch pruning with bounds from a dynamic program, and `pval_flexible()` chooses between the two according to the size of the instance.

Details

MultFourier computes the p-value $P\{T(Y) \geq T(x)\}$, $Y \sim \text{Multinomial}(N, p)$, of goodness-of-fit tests based on an additive statistic T : the log-likelihood ratio G^2 , Pearson's X^2 , the Cressie-Read power divergence, and the exact multinomial test (ordering by probability). It is aimed at small p-values, far in the tail, and at instances where the support of the multinomial is too large to sum over.

The package provides three functions:

- `pval_fourier`: writes the p-value as a Fourier series whose terms are values of the moment-generating function of the statistic, and sums it until an adaptive stopping rule is met. Its cost grows slowly with N and K ; its accuracy depends on the statistic (fast convergence for G^2 and the probability ordering, possibly none for X^2).
- `pval_exact`: exact p-value by branch pruning with bounds from a dynamic program, without enumerating the support. Feasible for a moderate number of categories.
- `pval_flexible`: chooses between the two according to the size of the support.

All three return an object of class "multfourier"; check its status component before using the p-value of `pval_fourier`.

Author(s)

Maintainer: Pablo Rivas <pablo.rivas.g@ug.uchile.cl>

Authors:

- Charles Thraves
- Felipe Subiabre

References

Subiabre, F. and Thraves, C. Efficient computation of p-values for multinomial tests. Manuscript.

See Also

Report bugs at <https://github.com/PabRvss/MultFourier-v2/issues>.

print.multfourier	<i>Print method for multfourier objects</i>
-------------------	---

Description

Prints the method, statistic, p-value, status and, for the Fourier method, the tilt, period half-width and number of terms.

Usage

```
## S3 method for class 'multfourier'
print(x, ...)
```

Arguments

x	Object of class "multfourier" returned by pval_fourier , pval_exact or pval_flexible .
...	Additional arguments (currently unused).

Value

Invisibly returns x.

pval_exact	<i>Exact multinomial test p-value</i>
------------	---------------------------------------

Description

Computes the exact p-value $P\{T(Y) \geq T(x)\}$, $Y \sim \text{Multinomial}(N, p)$, of a multinomial goodness-of-fit test, up to floating-point rounding. Instead of enumerating the support, it prunes whole groups of outcomes using bounds on the statistic (see Details). `pval_exhaustive()` is a deprecated alias.

Usage

```
pval_exact(
  x,
  p = rep(1/length(x), length(x)),
  stat = c("llr", "pd", "chi2", "pmf"),
  lambda = NULL,
  n_threads = min(2L, parallel::detectCores()),
  max_time = NULL,
  engine = c("speedup", "standard"),
  verbose = FALSE,
  ...
)

pval_exhaustive(...)
```

Arguments

x	Vector of observed counts, one per category (non-negative integers, without missing values). Their sum is the number of trials N .
p	Vector of category probabilities under the null hypothesis, all strictly positive. If they do not sum to 1 they are normalized to $p / \text{sum}(p)$, with a warning; the normalized vector is the one stored in the result. Default: equiprobable categories.
stat	Test statistic: "llr" (log-likelihood ratio G^2 , the default), "chi2" (Pearson's X^2), "pd" (Cressie–Read power divergence with parameter λ) or "pmf" (exact multinomial test, which orders outcomes by their probability). See Details.
lambda	Parameter λ of the power-divergence statistic; used only when <code>stat = "pd"</code> , and then required (there is no default); a warning is issued if it is given for another statistic. Cressie and Read (1984) recommend $\lambda = 2/3$. $\lambda = -1$ is not supported.
n_threads	Number of threads (positive integer, capped at the number of available cores). Default <code>min(2L, parallel::detectCores())</code> .
max_time	Optional time limit in seconds. NULL (default) means no limit. The limit is only checked between blocks of values of the first category, so a run can exceed it by a wide margin; if it is reached, <code>pval</code> is NA and <code>status</code> is 1.
engine	"speedup" (default): branch pruning with bounds from a dynamic program, described in Details. "standard": the earlier algorithm, which enumerates the first $K - 2$ categories and solves the last two by bisection.
verbose	Logical; if TRUE, prints progress information. Default FALSE.
...	Advanced options passed to the C engine. Unknown names produce a warning.

Details

The statistics are those of [pval_fourier](#). Because the statistic is additive, a dynamic program over the $(N + 1)K$ states (category, trials left) first computes the exact minimum and maximum that the remaining categories can add to it, at cost $O(KN^2)$. Counts are then assigned one category at a time, in decreasing order of probability; whenever the bounds show that every completion of a partial assignment is (or none is) at least as extreme as the observed data, its whole probability is added (or it is discarded) without visiting it. Only partial assignments near the boundary $T(y) = T(x)$ are expanded.

The cost still grows quickly with the number of categories K . As a rough guide, in our tests with the log-likelihood ratio on 8 threads, instances with $K \leq 6$ and $N \leq 1000$ took under five minutes, while $K \geq 8$ with $N \geq 500$ did not finish within an hour. For large instances use [pval_fourier](#).

Value

An object of class "multfourier", as described in [pval_fourier](#), with `method = "exact"`. The Fourier-specific components (`gamma`, `time_gamma`, `W`, `n_terms` and the four diagnostics) are NA. `status` is 0 when the computation finished and 1 when `max_time` was reached.

See Also

[pval_fourier](#), [pval_flexible](#).

Examples

```
p <- c(0.1, 0.2, 0.3, 0.4)
x <- c(35, 30, 60, 75)

pval_exact(x, p, stat = "llr")
pval_exact(x, p, stat = "chi2")$pval
pval_exact(x, p, stat = "pmf")$pval
```

pval_flexible

Multinomial test p-value, choosing the method automatically

Description

Computes the p-value with `pval_exact` when the instance is small and with `pval_fourier` otherwise.

Usage

```
pval_flexible(
  x,
  p = rep(1/length(x), length(x)),
  stat = c("llr", "pd", "chi2", "pmf"),
  lambda = NULL,
  enum_cutoff = 10,
  undersampling = 1,
  rel_eps = 0.001,
  B = 50,
  max_terms = 10000,
  max_time = NULL,
  avg_window = 0,
  avg_flat = FALSE,
  n_threads = min(2L, parallel::detectCores()),
  precision = c("double", "double-double"),
  precompute = TRUE,
  engine = c("speedup", "standard"),
  verbose = FALSE,
  ...
)
```

Arguments

x	Vector of observed counts, one per category (non-negative integers, without missing values). Their sum is the number of trials N .
p	Vector of category probabilities under the null hypothesis, all strictly positive. If they do not sum to 1 they are normalized to $p / \text{sum}(p)$, with a warning; the normalized vector is the one stored in the result. Default: equiprobable categories.
stat	Test statistic: "llr" (log-likelihood ratio G^2 , the default), "chi2" (Pearson's X^2), "pd" (Cressie–Read power divergence with parameter λ) or "pmf" (exact multinomial test, which orders outcomes by their probability). See Details.

lambda	Parameter λ of the power-divergence statistic; used only when <code>stat = "pd"</code> , and then required (there is no default); a warning is issued if it is given for another statistic. Cressie and Read (1984) recommend $\lambda = 2/3$. $\lambda = -1$ is not supported.
enum_cutoff	Threshold on $\log_{10}$ of the support size $\binom{N+K-1}{K-1}$ (the number of possible count vectors). Below it the exact method is used, at or above it the Fourier method. Default 10. See Details.
undersampling	Positive factor dividing the half-width W of the Fourier period. The default 1 uses the smallest W for which the series is exact. Values below 1 enlarge W (still exact, but more terms are needed); values above 1 shrink W below that minimum, so the series converges to a wrong value (aliasing). Intended for experimentation only.
rel_eps	Relative tolerance of the stopping rule (≥ 0). Default 0.001. It controls how stable the partial sums must be before the series is truncated; it is <i>not</i> a bound on the error of the returned p-value.
B	Non-negative integer used by the term-size stopping rule: the series stops after B consecutive terms each smaller, relative to the partial sum, than <code>rel_eps / B</code> . Default 50; B = 0 uses an internal value.
max_terms	Maximum number of series terms (positive integer). Default 10000. If it is reached before the stopping rule is met, status is 2.
max_time	Optional time limit in seconds, passed to the chosen method.
avg_window	Fraction (between 0 and 1) of the last partial sums to average into the reported value. Default 0 (no averaging). When positive, it replaces the extrapolated value of <code>engine = "speedup"</code> ; averaging is a linear filter that can bias the result and is usually less accurate than the default.
avg_flat	Logical; with <code>avg_window > 0</code> , TRUE gives equal weights and FALSE (default) weights that grow linearly towards the last partial sum.
n_threads	Number of threads, passed to whichever method is chosen. Default <code>min(2L, parallel::detectCores())</code> .
precision	Floating-point precision of the term evaluator: "double" (default) or "double-double". Only used with <code>engine = "standard"</code> ; ignored by the default engine.
precompute	Logical; if TRUE (default), precomputes the FFT plan of the term evaluator. Only used with <code>engine = "standard"</code> .
engine	"speedup" (default) or "standard", passed to the chosen method.
verbose	Logical; if TRUE, prints the partial sums and diagnostic information while running. Default FALSE.
...	Advanced options passed to the C engine, for example <code>print_freq</code> (how often partial sums are printed when <code>verbose = TRUE</code> ; default 100) or <code>extrapolate</code> (0: no extrapolation; 1: report the extrapolated value; 2: also stop on it, the default of <code>engine = "speedup"</code>). Unknown names produce a warning.

Details

The cost of `pval_exact` depends strongly on the observed counts, not only on the support size, and it is highest far in the tail, where small p-values are computed. With counts drawn from the null hypothesis it takes about one second on 2 threads at a support of 10^{12} to 10^{13} outcomes (log-likelihood ratio, $K = 5, 6, 7$); with counts far in the tail it can take tens of seconds at a support of 10^{11} (for example $K = 6$, $N = 400$, $p \approx 10^{-33}$). The default `enum_cutoff = 10` is a simple, conservative value that keeps the exact method for instances where it is fast in both situations.

When the Fourier method is chosen and does not meet its stopping rule (`status != 0`), the result is returned as is; there is no automatic fallback to the exact method.

Value

An object of class "multfourier", as described in [pval_fourier](#); its method component ("exact" or "fourier") shows which method was used.

See Also

[pval_fourier](#), [pval_exact](#).

Examples

```
# Small instance (K = 4, N = 200): exact method
pval_flexible(c(35, 30, 60, 75), c(0.1, 0.2, 0.3, 0.4))$method

# Large instance (K = 8, N = 2000, about 10^19 outcomes): Fourier method
fit <- pval_flexible(c(240, 260, 250, 270, 230, 250, 245, 255))
fit$method
fit$status
```

pval_fourier

Multinomial test p-value by Fourier series inversion

Description

Computes the p-value of a multinomial goodness-of-fit test,

$$p = P\{T(Y) \geq T(x)\}, \quad Y \sim \text{Multinomial}(N, p), \quad N = \sum_k x_k,$$

for an additive test statistic T , by summing a truncated Fourier series whose terms are values of the moment-generating function of the statistic. The series is truncated by an adaptive stopping rule, so the result is an approximation whose accuracy depends on the statistic and the instance; always check the returned status.

Usage

```
pval_fourier(
  x,
  p = rep(1/length(x), length(x)),
  stat = c("llr", "pd", "chi2", "pmf"),
  lambda = NULL,
  undersampling = 1,
  rel_eps = 0.001,
  B = 50,
  max_terms = 10000,
  max_time = NULL,
  avg_window = 0,
  avg_flat = FALSE,
  n_threads = min(2L, parallel::detectCores()),
  precision = c("double", "double-double"),
```

```

precompute = TRUE,
engine = c("speedup", "standard"),
verbose = FALSE,
...
)

```

Arguments

<code>x</code>	Vector of observed counts, one per category (non-negative integers, without missing values). Their sum is the number of trials N .
<code>p</code>	Vector of category probabilities under the null hypothesis, all strictly positive. If they do not sum to 1 they are normalized to $p / \text{sum}(p)$, with a warning; the normalized vector is the one stored in the result. Default: equiprobable categories.
<code>stat</code>	Test statistic: "llr" (log-likelihood ratio G^2 , the default), "chi2" (Pearson's X^2), "pd" (Cressie–Read power divergence with parameter λ) or "pmf" (exact multinomial test, which orders outcomes by their probability). See Details.
<code>lambda</code>	Parameter λ of the power-divergence statistic; used only when <code>stat = "pd"</code> , and then required (there is no default); a warning is issued if it is given for another statistic. Cressie and Read (1984) recommend $\lambda = 2/3$. $\lambda = -1$ is not supported.
<code>undersampling</code>	Positive factor dividing the half-width W of the Fourier period. The default 1 uses the smallest W for which the series is exact. Values below 1 enlarge W (still exact, but more terms are needed); values above 1 shrink W below that minimum, so the series converges to a wrong value (aliasing). Intended for experimentation only.
<code>rel_eps</code>	Relative tolerance of the stopping rule (≥ 0). Default 0.001. It controls how stable the partial sums must be before the series is truncated; it is <i>not</i> a bound on the error of the returned p-value.
<code>B</code>	Non-negative integer used by the term-size stopping rule: the series stops after B consecutive terms each smaller, relative to the partial sum, than <code>rel_eps / B</code> . Default 50; $B = 0$ uses an internal value.
<code>max_terms</code>	Maximum number of series terms (positive integer). Default 10000. If it is reached before the stopping rule is met, status is 2.
<code>max_time</code>	Optional time limit in seconds. NULL (default) means no limit. When it is reached, status is 1.
<code>avg_window</code>	Fraction (between 0 and 1) of the last partial sums to average into the reported value. Default 0 (no averaging). When positive, it replaces the extrapolated value of <code>engine = "speedup"</code> ; averaging is a linear filter that can bias the result and is usually less accurate than the default.
<code>avg_flat</code>	Logical; with <code>avg_window > 0</code> , TRUE gives equal weights and FALSE (default) weights that grow linearly towards the last partial sum.
<code>n_threads</code>	Number of threads used to evaluate the series terms (positive integer, capped at the number of available cores). Default <code>min(2L, parallel::detectCores())</code> .
<code>precision</code>	Floating-point precision of the term evaluator: "double" (default) or "double-double". Only used with <code>engine = "standard"</code> ; ignored by the default engine.
<code>precompute</code>	Logical; if TRUE (default), precomputes the FFT plan of the term evaluator. Only used with <code>engine = "standard"</code> .

engine	"speedup" (default) or "standard". See Details.
verbose	Logical; if TRUE, prints the partial sums and diagnostic information while running. Default FALSE.
...	Advanced options passed to the C engine, for example print_freq (how often partial sums are printed when verbose = TRUE; default 100) or extrapolate (0: no extrapolation; 1: report the extrapolated value; 2: also stop on it, the default of engine = "speedup"). Unknown names produce a warning.

Details

Statistics. All supported statistics are additive, $T(y) = \sum_k f_k(y_k)$, which is what the method requires. With $E_k = Np_k$:

- "llr": $G^2 = 2 \sum_k y_k \log(y_k/E_k)$.
- "chi2": $X^2 = \sum_k (y_k - E_k)^2/E_k$.
- "pd": $2I^\lambda = \frac{2}{\lambda(\lambda+1)} \sum_k y_k [(y_k/E_k)^\lambda - 1]$ (Cressie and Read, 1984); $\lambda \rightarrow 0$ gives G^2 and $\lambda = 1$ gives X^2 .
- "pmf": the p-value is $P\{P(Y) \leq P(x)\}$, the total probability of the outcomes that are at most as likely as the observed one.

Which statistics converge. The p-value is written exactly as a Fourier series for every additive statistic, but the number of terms needed for a given accuracy depends on the statistic. For "llr" and "pmf" the series typically converges in a few hundred to a few thousand terms. For "chi2", and often for "pd" unless λ is close to 0 ($\lambda = 2/3$ has failed on real data), the values of the statistic lie on or near a lattice, the terms do not decay, and the series can fail to converge within max_terms (status = 2) or even return a negative value. In that case use [pval_exact](#) when it is feasible.

Stopping rule and reported value. With engine = "speedup" the partial sums are accelerated with Wynn's epsilon algorithm (in the safeguarded form of QUADPACK's DQELG) and smoothed with a running median of five. The series stops as soon as either (i) B consecutive terms are each smaller than rel_eps / B relative to the partial sum, or (ii) the smoothed extrapolated values vary by at most a relative rel_eps over a trailing window covering the last half of the terms. The reported p-value is the last smoothed extrapolated value. With engine = "standard" only rule (i) is used and the reported p-value is the last partial sum.

Engines. "speedup" evaluates the terms with a Poissonized Cauchy-integral evaluator, chooses the exponential tilt gamma by a Newton saddlepoint search, computes the range of the statistic in closed form, and uses the extrapolation described above. "standard" is the earlier implementation: polynomial convolution evaluated by FFT, a grid search for the tilt, a dynamic program for the range, and no extrapolation. Both compute the same series.

Value

An object of class "multfourier": a list with components

- x, p** The input counts and null probabilities (p after normalization, if it did not sum to 1).
- pval** The computed p-value.
- stat** Name of the test statistic, e.g. "Log-Likelihood Ratio" or "power-divergence (lambda = 0.6667)".
- method** "fourier".
- time** Total wall-clock time in seconds.

gamma The exponential tilt γ applied to the statistic so that the series terms stay within floating-point range (a saddlepoint choice).

time_gamma Time in seconds spent choosing gamma.

W Half-width of the Fourier period, $\max(s_{\max}, -s_{\min})$ over the range $[s_{\min}, s_{\max}]$ of the statistic minus its observed value, divided by undersampling.

n_terms Number of series terms summed.

status 0: the stopping rule was met (the message reads "Converged"); 1: max_time was reached; 2: max_terms was reached without meeting the stopping rule, so pval is unreliable; 3: the search for gamma failed; 4: numerical overflow in the evaluation of the terms. With a status other than 0, pval is the last value reached or NA.

message Human-readable version of status.

err_rel_ext Relative spread of the smoothed extrapolated values over the final window, i.e. the quantity that rule (ii) compares with rel_eps. When the series stopped by rule (i) it can be larger than rel_eps.

err_rel_est Estimate of the relative truncation error of the raw partial sum (not of the extrapolated value that is reported), based on a random-phase argument; in our tests it tends to underestimate the error.

n_eff Effective number of support points implied by the size of the late terms; used by err_rel_est.

nu_max_ratio Largest size of the late transform values relative to the first one; values near 1 mean the terms are not decaying (a lattice-like statistic).

The last four components are diagnostics of the series; they are heuristics, not error bounds. The names Gamma and Time_gamma used by earlier versions still work, with a deprecation warning.

References

- Subiabre, F. and Thraves, C. Efficient computation of p-values for multinomial tests. Manuscript.
- Cressie, N. and Read, T. R. C. (1984). Multinomial goodness-of-fit tests. *Journal of the Royal Statistical Society, Series B*, 46(3), 440–464.

See Also

[pval_exact](#) for the exact p-value, [pval_flexible](#) to choose between the two automatically.

Examples

```
p <- c(0.1, 0.2, 0.3, 0.4)
x <- c(35, 30, 60, 75) # N = 200

fit <- pval_fourier(x, p, stat = "llr")
fit
fit$pval
fit$status          # 0: stopping rule met

# Compare with the exact p-value
pval_exact(x, p, stat = "llr")$pval

# Power divergence with the Cressie-Read parameter
pval_fourier(x, p, stat = "pd", lambda = 2/3)$pval
```

Index

* **package**

 MultFourier-package, [2](#)

MultFourier (MultFourier-package), [2](#)

MultFourier-package, [2](#)

print.multfourier, [3](#)

pval_exact, [2](#), [3](#), [3](#), [5–7](#), [9](#), [10](#)

pval_exhaustive (pval_exact), [3](#)

pval_flexible, [2–4](#), [5](#), [10](#)

pval_fourier, [2–5](#), [7](#), [7](#)